

Junior Developer · From Service Development to Network & Security · [GitHub profile](#) · [Portfolio site](#)

For each project: my role and scope, then the situation, my judgment, what I did and the result, written as facts that can be checked.

ExplainSOC — Security-AI Performance Claim Checker (ALEPH Assignment 13) 2026.09

Role Planning · Front-end implementation · User observation · Validation · Deployment

Scope 15 reading rules, conclusion card, paper lab, six user sessions, reproducible ZIP, GitHub Pages

Strength Self-regulation

Situation Security AI product brochures lead with numbers like '99% accuracy' but rarely say which test produced them. In my Assignment 10 paper, the same XGBoost scored Macro F1 0.9979 on a randomly split test and 0.3871 on a test holding only attacks absent from training.

Judgment I chose not to build scores, grades or pass marks that judge a number good or bad. Instead, the tool checks the test conditions behind a number first, then turns missing evidence into questions to ask the vendor.

Action I wrote 15 reading rules that find missing test conditions in brochure sentences and score tables and turn them into vendor questions, and built a conclusion card and a paper lab for changing the paper's experiments directly. Rather than adding features, I recorded first-time users six times and fixed each place they got stuck. The app runs entirely in the browser, with no server.

Result A fellow trainee reached the vendor questions on a phone unaided; 187 tests gate every deploy.

Stack React 19 · TypeScript · Vite 8 · Vitest · Playwright

[Try the app](#) · [Public repository](#)

CLOV — Team Project 2026.06 - 2026.08

Role Team lead · Full-stack developer · Team of four

Scope Service architecture, React UI and Spring Boot API, deployment on Ubuntu, Docker, Nginx and HTTPS, issue tracking and code review

Strength Interpersonal skills

Situation Only two people came to the first meeting of a team that included three non-majors, and code produced with different AI tools often clashed.

Judgment I built a prototype that showed features before explaining them, and made an issue before work and a run check and code review before merge the team's shared standard.

Action I built a working prototype first and ran a flow of writing an issue, checking that it runs, and merging only after code review.

Result By August 4, 2026 the team logged 152 issues and 266 pull requests (72 reviewed by me) and served the app over HTTPS.

Stack React · Spring Boot · Docker · Nginx · OAuth

[Front-end repository](#) · [Back-end repository](#)

Network Intrusion Detection Research (ALEPH Assignment 10)

2026.09

Role Research design · Implementation · Validation · Paper writing

Scope Research design, data audit, three-model comparison, SHAP stability checks, paper and reproducible materials

Strength Self-motivation

Situation I needed to check whether high detection scores on a random split actually generalize to attacks the model has never seen.

Judgment I designed a random split and a date-based unseen-attack split separately, and compared three models' performance and SHAP explanation stability under the same conditions.

Action I audited 2,830,743 CICIDS2017 records, compared three models on the random and unseen-attack splits, and checked the stability of SHAP explanations.

Result Submitted a paper with reproducible materials: XGBoost Macro F1 fell from 0.9979 to 0.3871.

Stack Python · scikit-learn · XGBoost · SHAP

[Read the paper](#) · [Public repository](#)

Passkey-Protected Private Area (ALEPH Assignment 8)

2026.09

Role WebAuthn authentication design · Implementation · Validation · Deployment

Scope WebAuthn registration and sign-in, session, Origin and CSRF checks, database, public deployment

Strength Self-regulation

Situation I had to keep a public introduction page open while letting only myself see private material, without a password.

Judgment Instead of passwords, I decided to confirm identity with a device passkey and to check the boundary between public and private content at every request.

Action I implemented WebAuthn registration and sign-in, and made each one-time challenge usable only once in the database. Server sessions, Origin and CSRF checks, and a guard against deleting the last passkey are all enforced at the request level.

Result Met all 47 acceptance criteria and passed 50 tests; zero password fields on public pages, 401 for private requests without a session.

Stack Java 25 · Spring Boot · WebAuthn4J · PostgreSQL · Vercel

[Public repository](#)

Plan-Do-See Diary Account Authentication (ALEPH Assignment 7)

2026.09

Role Authentication design · Full-stack implementation · Validation · Deployment

Scope Argon2id password storage, JWTs bound to server sessions with refresh-token rotation, per-account data isolation, migration of existing records, Docker deployment on Render

Strength Self-regulation

Situation I had to turn a Plan-Do-See diary that started as a public app into one based on personal accounts. A JWT checked only by its signature stays valid after logout until it expires, and hiding the diary behind a login screen alone would still leave a path to read other accounts' records.

Judgment I bound each access JWT to a server session row so every request checks that the session is alive and who owns the data, and treated a reused, already-rotated refresh token as possible theft that revokes the whole token family.

Action I stored passwords with Argon2id and limited lists, single records, statistics and exports to the user ID confirmed by the server. Existing records were backed up and moved to the new account, and changing the password revokes every existing session.

Result Passed 319 back-end and 76 front-end tests, with request/response evidence that access to other accounts' data and reused tokens are rejected.

Stack React · TypeScript · Flask · PostgreSQL · Argon2id · Docker

[Try the service \(slow first load\)](#) · [Authentication guide](#) · [Public repository](#)

AI Handoff Without Sharing the Conversation (ALEPH Assignment 5)

2026.08 - 2026.09

Role Experiment design · Front-end implementation · Validation · Deployment

Scope Ten fixed checks, a slider that overlays the original and finished card, a seven-part handoff document, unit and E2E regression tests, GitHub Pages deployment

Strength Self-motivation

Situation Every time I switched AI tools I had to explain the same context again, and it had never been tested whether the next AI could safely continue the same project without the conversation.

Judgment Instead of racing for speed, I fixed the feature scope and ten fixed checks first, and tested whether the next AI could take over from code, checks and a handoff document alone, without the conversation.

Action I first set the rule that a check not actually run is never recorded as passed, and had the first AI implement only checks 1–3 and stop as planned. The next AI read only the handoff document and finished checks 4–10 and the full regression suite. When the next AI read the first AI's work log before the prohibition rules, I did not hide that procedural violation behind the passing result; I left it in the work log and on the public page.

Result Passed all 10 fixed checks; 85 unit and 38 E2E tests stayed green with no human code fixes.

Stack React 19 · Vite 8 · Canvas API · Playwright

[Try the site](#) · [Handoff results](#) · [Public repository](#)

Security Board with Alerts and Privilege Revocation (course lab)

2026.09

Role Security feature design and implementation · Automation integration · Validation

Scope Role-based access control with admin safeguards, an n8n sign-in alert bot with three messenger channels, porting the course's automatic over-privilege revocation code, Graylog SYN flood detection

Strength Self-regulation

Situation I had to add role-based access control to a board from class and port the course's code that automatically revokes excess privileges. If automatic revocation misfired, it could demote even the administrators and leave no one able to manage members.

Judgment The more automatic a revocation feature is, the more it must first prevent administrators from locking themselves out, so I applied the same rules that forbid self-demotion, self-deletion and demoting the last administrator to the revocation bot.

Action I split the revocation bot into three stages (report only, notify Graylog, actually demote) so it removes privileges only when given the option, and kept evidence captures and alert senders from exposing API keys or webhook addresses on screen or in error messages. In the SYN flood lab I did not stop at seeing the alert arrive; I compared it with the run record and fixed a bug where the attack was marked as allowed and the real source and count were missing.

Result Passed 45 tests; a 35,033-packet SYN flood from Kali reached Graylog, n8n, three messengers and the board log.

Stack Flask · MySQL · JWT · n8n · Graylog · Docker

[Public repository](#)

GAME PULSE Data Board (ALEPH Assignment 4)

2026.08 - 2026.09

Role Data collection design · Front-end implementation · Validation · Deployment

Scope Collecting and storing four Steam feeds (concurrent players, sales, discounts, streams), a React 19 UI, failure reproduction and recovery, GitHub Pages deployment

Strength Self-regulation

Situation When numbers collected from several sources share one screen, it becomes unclear when and where each value came from and what is shown when collection fails.

Judgment I chose Steam over Stack Overflow and narrowed it to four axes (concurrent players, sales, discounts and streams). When collection fails, the board shows the last good value and its time instead of inventing an unknown one.

Action I stored each value with its source, measurement time, data state and calculation basis, and tested recovery by reproducing five kinds of failure, including timeouts.

Result Passed all 317 regression tests with an audit trail for all 35 assignment conditions.

Stack React 19 · Vite 8 · Node 22 · GitHub Actions · GitHub Pages

[Try the site](#) · [Public repository](#)

SOC SHIFT:30 Alert-Triage Game (ALEPH Assignment 2)

2026.08

Role Planning · Front-end implementation · Testing · Deployment

Scope Thirty-second triage rules, React 18 and TypeScript, keyboard, mouse and touch controls, serverless deployment on GitHub Pages

Strength Self-motivation

Situation To let people briefly experience security-monitoring judgment, a single accuracy score does not reveal what they actually got wrong.

Judgment I judged that counting false positives and false negatives separately keeps a round survived by blocking everything from scoring the same as a round that sorted alerts correctly.

Action I built a 30-second shift where each alert must be passed or blocked within three seconds, and the results screen counts false positives and false negatives separately as bars.

Result Serverless build that passed 291 tests; playable from a link with no install or sign-in.

Stack React 18 · TypeScript · Vite 8

[Play the game](#) · [Public repository](#)